IaC / DEVOPS LEARNING SERIES

Terraform Complete Roadmap

Azure + AWS, with Interview Q&A

27 modules: fundamentals, Azure, AWS, modules, state, CI/CD, capstone
12 hands-on portfolio projects with real Terraform code
Azure <-> AWS concept mapping for fast multi-cloud fluency
40+ interview questions with model answers

Verify exact resource arguments, provider versions and backend syntax against current Terraform documentation.

How to Use This Roadmap

This roadmap teaches Terraform once, then has you practice the same core concepts twice: first on Azure (building on the networking you already know), then on AWS (by explicitly mapping every concept across). The target end-state:

```
GitHub -> Terraform -> Cloud Provider
                         +-- Azure: Resource Group, VNet, Subnets, NSG, VM, Load Balancer, NAT, AKS
                         +-- AWS:   VPC, Subnets, Route Tables, Security Groups, EC2, ALB, NAT Gateway, EKS
                         -> Remote State -> Modules -> GitHub Actions -> CI/CD -> Production Infrastructure
```

The ideal ratio is roughly 30% learning and 70% hands-on. For every module below, read the explanation once, then actually type and run the practical example against a real (free-tier) account before moving on.

27 Modules at a Glance

#	Module	Phase
M00	Prerequisites: Linux, Git & Networking	Prerequisites
M01	What is Terraform? (Concepts)	Terraform Fundamentals
M02	Installing Terraform & Your First Project	Terraform Fundamentals
M03	HCL Syntax: Blocks, Arguments & Attributes	HCL Deep Dive
M04	Terraform Core Block Types	HCL Deep Dive
M05	The Terraform Workflow: init -> fmt -> validate -> plan -> apply	Terraform Workflow
M06	PROJECT 1 - Azure Resource Group	Azure Terraform Projects
M07	PROJECT 2 - Azure VNet, Subnets & NSGs	Azure Terraform Projects
M08	PROJECT 3 - Azure Linux VM (First Portfolio Project)	Azure Terraform Projects
M09	PROJECT 4 - Azure 3-Tier Architecture	Azure Terraform Projects
M10	Variables: Types, Validation & Sensitive Data	Variables, Locals, Outputs & Data Sources
M11	Locals, Outputs & Data Sources	Variables, Locals, Outputs & Data Sources
M12	Loops: count vs for_each	Loops & Conditionals
M13	Conditional Expressions & Dynamic Blocks	Loops & Conditionals
M14	Terraform State: Concepts & Local State Commands	Terraform State
M15	Remote State: Azure Storage & AWS S3 Backends	Terraform State
M16	Terraform Modules: Structure & Reuse	Modules
M17	PROJECT 5 - Azure Multi-Environment Platform	Modules
M18	PROJECT 6 - Azure Networking + Terraform Troubleshooting Lab	Modules
M19	Azure to AWS Concept Mapping	AWS Terraform Projects
M20	PROJECT 7 - AWS VPC + Subnets + Routing	AWS Terraform Projects
M21	PROJECT 8 - AWS EC2 + User Data (Nginx) & 3-Tier Architecture	AWS Terraform Projects
M22	Terraform + Kubernetes (AKS / EKS) and the Terraform + Ansible Boundary	Terraform + Kubernetes & Ansible
M23	Provisioners, Workspaces & Terraform Import	Provisioners, Workspaces & Import
M24	Debugging, Security & Secrets Management	Debugging, Security & CI/CD
M25	Terraform CI/CD, Linting & Security Scanning	Debugging, Security & CI/CD
M26	CAPSTONE - Multi-Cloud DevOps Infrastructure Platform	Capstone

Recommended Learning Order

Do not work through the modules by watching everything linearly without building. Use these 8 levels as your mental map of how the modules fit together:

Level	Covers	Modules
Level 1 - Fundamentals	Concepts, IaC, Terraform vs Ansible, HCL, providers, resources, variables, outputs, locals	M1-M4, M10-M11
Level 2 - Workflow	init, fmt, validate, plan, apply, destroy	M5
Level 3 - Expressions	functions, interpolation, conditions, count, for_each, dynamic blocks	M12-M13
Level 4 - State	state, backend, remote state, state locking, import, state commands	M14-M15, M23
Level 5 - Modules	module structure, inputs, outputs, registry modules, custom modules	M16-M17
Level 6 - Azure	Resource Group -> VNet -> Subnets -> NSG -> VM -> Load Balancer -> 3-tier -> AKS	M6-M9, M18, M22
Level 7 - AWS	VPC -> Subnets -> Route Tables -> IGW -> NAT -> Security Groups -> EC2 -> ALB -> RDS -> EKS	M19-M22
Level 8 - DevOps	GitHub -> GitHub Actions -> Terraform -> security scanning -> plan -> approval -> apply	M24-M26

Azure <-> AWS Concept Map

Learn Terraform's core once, then use this table to translate every Azure concept you already know into its AWS equivalent, instead of learning AWS from zero.

Concept	Azure	AWS
Resource grouping	Resource Group	No direct equivalent (use tags/naming)
Network	VNet	VPC
Network subdivision	Subnet	Subnet
Firewall (instance level)	Network Security Group (NSG)	Security Group
Firewall (subnet level)	NSG on subnet	Network ACL (NACL)
Routing	Route Table + UDR	Route Table
Public address	Public IP	Elastic IP / Public IP
Virtual machine	Virtual Machine (VM)	EC2 instance
Load balancing	Azure Load Balancer / App Gateway	ALB / NLB
Outbound-only gateway	NAT Gateway	NAT Gateway
Identity & access	Microsoft Entra ID + Azure RBAC	IAM
Managed Kubernetes	AKS	EKS
Managed relational DB	Azure Database for PostgreSQL/MySQL	RDS
Secret storage	Azure Key Vault	Secrets Manager / SSM Parameter Store
Object storage	Blob Storage	S3
Remote state backend	Storage Account + Blob Container	S3 bucket (+ native state locking)

12 Portfolio Projects

#	Project	Cloud	Difficulty	Module
1	Resource Group + VM	Azure	Beginner	6, 8
2	VNet + Subnets + NSG	Azure	Beginner	7
3	3-Tier Infrastructure	Azure	Intermediate	9
4	NVA + UDR Networking Lab	Azure	Intermediate	18
5	Multi-Environment IaC (dev/staging/prod)	Azure	Advanced	17
6	VPC + EC2	AWS	Beginner	20
7	3-Tier Architecture (ALB + RDS)	AWS	Intermediate	21
8	ALB + Auto Scaling + RDS	AWS	Advanced	21
9	AKS with Terraform	Azure	Advanced	22
10	EKS with Terraform	AWS	Advanced	22
11	Terraform + GitHub Actions Pipeline	Both	Advanced	25
12	Multi-Cloud Terraform Platform (Capstone)	Azure + AWS	Flagship	26

The 27 Modules: Detailed Roadmap

Each module has a goal, a plain-language explanation, the topics to study, a hands-on practical example, interview questions, and a concrete deliverable.

PHASE 0 - Prerequisites

M00 Prerequisites: Linux, Git & Networking

Goal: Confirm you have the baseline skills Terraform assumes, so the roadmap itself goes smoothly.

EASY EXPLANATION

Terraform issues API calls on your behalf and then waits for cloud resources to come up. If you can't already navigate Linux, use Git, and reason about IP/CIDR/routing, every error message in this roadmap will be twice as confusing. Spend a day confirming these first.

TOPICS TO STUDY

- Linux: ssh, systemctl, curl, wget, chmod, chown, grep, cat, vim/nano
- Git: init, add, commit, push, branch, merge
- Networking: IP, CIDR, Subnet, Route Table, Gateway, NAT, DNS, Firewall, Security Group/NSG, public vs private subnet
- Why this matters extra for you: you already work with Azure UDR/NVA networking, which is exactly what Terraform will let you version and automate

HANDS-ON PRACTICAL

```
# quick self-check commands
ssh -V
git --version
ip addr show
curl -I https://example.com
# CIDR self-test: what is the usable host range of 10.0.1.0/28?
```

Interview questions

Q1 What is CIDR notation and how do you calculate the number of usable IPs in a /24 vs a /28?

Q2 What is the difference between a public and a private subnet, and what component makes a subnet 'public'?

Deliverable: A one-page personal notes file confirming you can explain CIDR, routing and basic Git commands without looking them up.

PHASE 1 - Terraform Fundamentals

M01 What is Terraform? (Concepts)

Goal: Understand the mental model before writing any code: declarative infrastructure, desired state, providers and the plan/apply cycle.

EASY EXPLANATION

Instead of clicking through a portal or running imperative scripts step by step, you describe the infrastructure you WANT to exist (desired state) in a file. Terraform compares that description to what actually exists (its state) and works out the minimum set of API calls to get from one to the other. A 'provider' is the plugin that knows how to talk to a specific cloud's API (Azure, AWS, Docker, GitHub, etc).

TOPICS TO STUDY

- Infrastructure as Code (IaC) vs manual/imperative provisioning
- Declarative configuration vs imperative scripting
- Desired state vs current state vs real infrastructure
- Providers, resources and their dependency graph
- Terraform state as the source of truth Terraform itself uses
- The terraform graph: how Terraform decides create/update/destroy order

HANDS-ON PRACTICAL

```
Human
  |
Terraform Code (.tf files)   <- desired state, declarative
  |
Terraform Plan               <- diff against state
  |
Terraform Provider           <- plugin (azurerm, aws, docker...)
  |
Cloud API
  |
Infrastructure
```

Interview questions

Q1 Explain 'declarative' vs 'imperative' infrastructure management with an example.

Q2 What is a Terraform provider, and how is it different from a resource?

Q3 Terraform vs Ansible vs CloudFormation: how would you compare them in an interview?

Deliverable: A short written explanation (in your own words) of the human -> code -> plan -> provider -> API -> infrastructure flow.

M02 Installing Terraform & Your First Project

Goal: Get Terraform installed, verified, and lay out a clean project skeleton you will reuse for every project in this roadmap.

EASY EXPLANATION

Every Terraform project you ever build will use roughly the same four files: main.tf (the resources), variables.tf (inputs), outputs.tf (results you want to see or pass along) and terraform.tfvars (actual values for this run). Getting this skeleton right now saves confusion later.

TOPICS TO STUDY

- Installing Terraform (binary or package manager) and checking the version
- terraform -help and discovering subcommands
- Standard project layout: main.tf, variables.tf, outputs.tf, terraform.tfvars
- .gitignore for Terraform (never commit state files or secrets)

HANDS-ON PRACTICAL

```
terraform version
terraform -help

mkdir terraform-demo && cd terraform-demo
touch main.tf variables.tf outputs.tf terraform.tfvars
cat > .gitignore << 'EOF'
*.tfstate
*.tfstate.backup
.terraform/
*.tfvars
!example.tfvars
EOF
```

Interview questions

Q1 Why should terraform.tfstate and *.tfvars usually NOT be committed to Git?

Q2 What is the purpose of separating variables.tf from terraform.tfvars?

Deliverable: A terraform-demo/ folder with the standard 4-file skeleton, committed to Git with a correct .gitignore.

PHASE 2 - HCL Deep Dive

M03 HCL Syntax: Blocks, Arguments & Attributes

Goal: Read and write HashiCorp Configuration Language (HCL) fluently: the syntax every .tf file is written in.

EASY EXPLANATION

HCL has exactly three things to learn: blocks (containers, like a resource or a variable), arguments (key = value pairs inside a block) and attributes (values you can reference out of a resource once it exists, like azurerm_resource_group.main.name). Once these three clicks, the rest of Terraform is just learning provider resource types.

TOPICS TO STUDY

- Blocks: resource, provider, variable, output, data, module, terraform, locals
- Arguments: key = value pairs inside a block
- Attributes: referencing a value out of a resource after creation
- Comments: # or // for single line, /* */ for multi-line

HANDS-ON PRACTICAL

```
# a resource BLOCK with two ARGUMENTS
resource "azurerm_resource_group" "main" {
  name     = "rg-demo"
  location = "East US"
}
```

```
# referencing an ATTRIBUTE of the resource above
# azurerm_resource_group.main.name  ->  "rg-demo"
output "rg_name" {
  value = azurerm_resource_group.main.name
}
```

Interview questions

Q1 What is the difference between an argument and an attribute in Terraform?

Q2 How do you reference the ID of one resource from inside another resource block?

Deliverable: A one-page HCL cheat sheet in your own words: block / argument / attribute, with one example each.

M04 Terraform Core Block Types

Goal: Learn what each of the 7 core Terraform block types is for, before you touch a real cloud.

EASY EXPLANATION

Think of these seven blocks as the entire vocabulary of Terraform: 'terraform' configures Terraform itself (providers/version), 'provider' configures a specific cloud connection, 'resource' creates something, 'variable' takes an input, 'locals' computes a reusable value, 'output' prints/exports a value, and 'data' reads (but doesn't create) something that already exists.

TOPICS TO STUDY

- terraform { } - required_providers, required_version, backend
- provider { } - configures a specific cloud/service connection
- resource { } - declares something Terraform should create and manage
- variable { } - declares an input parameter
- locals { } - computed/derived values reused across the config
- output { } - values exposed after apply (for humans or other configs)
- data { } - reads existing infrastructure without managing it

HANDS-ON PRACTICAL

```
variable "location" {
  type    = string
  default = "East US"
}

locals {
  full_name = "rg-${var.location}-demo"
}

resource "azurerm_resource_group" "main" {
  name     = local.full_name
  location = var.location
}

output "resource_group_id" {
  value = azurerm_resource_group.main.id
}
```

Interview questions

Q1 Name the 7 core Terraform block types and what each is used for.

Q2 When would you use a 'data' block instead of a 'resource' block?

Deliverable: A diagram (hand-drawn or draw.io) showing how variable -> locals -> resource -> output connect in one file.

PHASE 3 - Terraform Workflow

M05 The Terraform Workflow: init -> fmt -> validate -> plan -> apply

Goal: Memorize and practice the exact sequence of commands used on every single Terraform run, in every job you'll ever have.

EASY EXPLANATION

init downloads the providers/modules your code needs. fmt rewrites your files into consistent style. validate checks syntax without touching any cloud. plan shows you EXACTLY what would change, without changing anything. apply actually performs those changes. Never skip plan before apply in a real environment.

TOPICS TO STUDY

- terraform init: downloads providers, initializes backend/modules
- terraform fmt: auto-formats .tf files to canonical style
- terraform validate: syntax and internal consistency check (no cloud calls)
- terraform plan: shows the diff between desired and current state
- terraform apply: executes the plan (creates/updates/destroys)
- terraform destroy: tears down everything Terraform manages in this config

HANDS-ON PRACTICAL

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
# when you are done experimenting:
terraform destroy
```

Interview questions

Q1 Walk through what happens, in order, when you run these five commands on a brand-new project.

Q2 Why is it good practice to save 'terraform plan -out=tfplan' and apply that exact file instead of running 'terraform apply' directly?

Deliverable: A short recording or screenshot sequence showing init -> plan -> apply -> destroy against a real (free-tier) resource.

PHASE 4 - Azure Terraform Projects

M06 PROJECT 1 - Azure Resource Group

Goal: Your very first real Terraform deployment: a single Azure Resource Group, end to end.

EASY EXPLANATION

This project has exactly one resource, on purpose. The goal isn't the resource group itself — it's proving you can go from provider configuration, through variables and outputs, to a real object appearing in Azure and being destroyed cleanly.

TOPICS TO STUDY

- Configuring the azurerm provider and the features {} block
- Authenticating Terraform to Azure (az login or a service principal)
- resource, variable and output working together end to end
- The Terraform lifecycle meta-argument (create_before_destroy, prevent_destroy, ignore_changes)

HANDS-ON PRACTICAL

```
# main.tf
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
}

provider "azurerm" {
  features {}
}

variable "rg_name"  { type = string  default = "rg-terraform-demo" }
variable "location" { type = string  default = "East US" }

resource "azurerm_resource_group" "main" {
  name     = var.rg_name
  location = var.location

  lifecycle {
    prevent_destroy = false   # set true in real production
  }
}

output "resource_group_id" {
  value = azurerm_resource_group.main.id
}
```

Interview questions

Q1 What does the lifecycle { prevent_destroy = true } setting protect you from?

Q2 How does Terraform authenticate to Azure, and what are the different ways to provide credentials?

Deliverable: PROJECT 1 complete: Resource Group created and destroyed via Terraform, with a screenshot of the Azure Portal and the CLI output.

M07 PROJECT 2 - Azure VNet, Subnets & NSGs

Goal: Build a small but real network: VNet with two subnets, each protected by its own NSG.

EASY EXPLANATION

This is where your existing Azure networking knowledge pays off. You are not learning new networking concepts here — you are learning how to EXPRESS networking concepts you already know as Terraform resources with explicit and implicit dependencies between them.

TOPICS TO STUDY

- azurerm_virtual_network and azurerm_subnet
- azurerm_network_security_group and its rules
- azurerm_subnet_network_security_group_association (explicit linking resource)
- Implicit dependencies (referencing an attribute) vs explicit depends_on
- CIDR planning for VNet + 2 subnets without overlap

HANDS-ON PRACTICAL

```
resource "azurerm_virtual_network" "main" {
  name                = "vnet-demo"
  address_space       = ["10.0.0.0/16"]
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
}

resource "azurerm_subnet" "web" {
  name                 = "snet-web"
  resource_group_name  = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.0.1.0/24"]
}

resource "azurerm_subnet" "app" {
  name                 = "snet-app"
  resource_group_name  = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.0.2.0/24"]
}

resource "azurerm_network_security_group" "web" {
  name                = "nsg-web"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name

  security_rule {
    name                       = "allow-http"
    priority                   = 100
    direction                  = "Inbound"
    access                     = "Allow"
    protocol                   = "Tcp"
    source_port_range          = "*"
    destination_port_range     = "80"
    source_address_prefix      = "*"
    destination_address_prefix = "*"
  }
}

resource "azurerm_subnet_network_security_group_association" "web" {
  subnet_id                 = azurerm_subnet.web.id
  network_security_group_id = azurerm_network_security_group.web.id
}
```

Interview questions

Q1 How does Terraform know to create the VNet before the subnet, without you writing depends_on?

Q2 Why is a separate 'association' resource needed to attach an NSG to a subnet in the azurerm provider?

Deliverable: PROJECT 2 complete: VNet with 2 subnets and 1 NSG correctly attached; verified in Azure Portal that rules apply.

M08 PROJECT 3 - Azure Linux VM (First Portfolio Project)

Goal: Provision a complete, SSH-reachable Linux VM: public IP, NIC, NSG and VM all wired together by Terraform.

EASY EXPLANATION

This is the first project where the OUTCOME matters, not just the resources: at the end you should be able to SSH into a machine that only exists because Terraform created it. That's a genuinely portfolio-worthy result.

TOPICS TO STUDY

- azurerm_public_ip, azurerm_network_interface
- Associating a public IP with a NIC, and a NIC with a VM
- azurerm_linux_virtual_machine and SSH key-based authentication
- Referencing outputs to get the VM's public IP for SSH

HANDS-ON PRACTICAL

```
resource "azurerm_public_ip" "main" {
  name                = "pip-demo"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  allocation_method   = "Static"
  sku                 = "Standard"
}

resource "azurerm_network_interface" "main" {
  name                = "nic-demo"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name

  ip_configuration {
    name                          = "internal"
    subnet_id                     = azurerm_subnet.web.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id          = azurerm_public_ip.main.id
  }
}

resource "azurerm_linux_virtual_machine" "main" {
  name                = "vm-demo"
  resource_group_name = azurerm_resource_group.main.name
  location            = azurerm_resource_group.main.location
  size                = "Standard_B1s"
  admin_username      = "azureuser"
  network_interface_ids = [azurerm_network_interface.main.id]

  admin_ssh_key {
    username   = "azureuser"
    public_key = file("~/.ssh/id_rsa.pub")
  }

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }
}

output "vm_public_ip" {
  value = azurerm_public_ip.main.ip_address
```

```
}
```

Interview questions

Q1 Why is a Standard SKU public IP typically required for a Standard Load Balancer/VM setup in Azure?

Q2 How does Terraform's file() function work, and why is it used here for the SSH key?

Deliverable: PROJECT 3 complete: 'ssh azureuser@<output-ip>' works, and you can destroy/recreate the VM on demand.

M09 PROJECT 4 - Azure 3-Tier Architecture

Goal: Combine everything so far into a realistic multi-tier design: Load Balancer -> Web tier -> App tier -> DB tier.

EASY EXPLANATION

Production infrastructure is rarely one resource — it's a graph of many resources that depend on each other in specific ways. This project is deliberately more complex so you build the habit of managing many interdependent resources cleanly instead of one giant unreadable file.

TOPICS TO STUDY

- azurerm_lb, azurerm_lb_backend_address_pool, azurerm_lb_rule
- Multiple VMs across multiple subnets (web + app tiers)
- azurerm_network_interface_backend_address_pool_association
- A private DB subnet with no public IP path
- Organizing a growing main.tf into multiple .tf files (network.tf, compute.tf, database.tf)

HANDS-ON PRACTICAL

```
Internet
   |
Load Balancer (azurerm_lb + rule on port 80)
   |
  +---------------------+
  |                     |
Web Subnet            Web Subnet
  |                     |
 App VM                App VM
  |                     |
  +----------+----------+
             |
        DB Subnet (no public IP, NSG denies internet inbound)
             |
         Database
```

Interview questions

Q1 How would you split this project's .tf files to keep it maintainable as it grows?

Q2 What NSG rule design ensures the DB subnet is reachable only from the App tier, never from the internet?

Deliverable: PROJECT 4 complete: traffic to the Load Balancer's public IP reaches a VM; DB subnet proven unreachable from the internet.

PHASE 5 - Variables, Locals, Outputs & Data Sources

M10 Variables: Types, Validation & Sensitive Data

Goal: Go beyond string/number variables: learn every Terraform variable type and how to validate and protect them.

EASY EXPLANATION

A variable isn't just 'a value you can change' — Terraform lets you constrain its type, give it a description for teammates, validate its contents before anything runs, and mark it sensitive so it never gets printed in plan/apply output or logs.

TOPICS TO STUDY

- Variable types: string, number, bool, list, set, map, object, tuple
- default, description, type and validation arguments
- sensitive = true and where its value still ends up (state file!)
- Passing variables: -var, -var-file, terraform.tfvars, TF_VAR_ environment variables

HANDS-ON PRACTICAL

```
variable "environment" {
  type        = string
  description = "Deployment environment name"
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be dev, staging or prod."
  }
}

variable "admin_password" {
  type      = string
  sensitive = true
}

# terraform.tfvars
environment = "dev"

# or via environment variable (useful in CI/CD, avoids files with secrets)
# export TF_VAR_admin_password="S3cret!"
```

Interview questions

Q1 Does marking a variable 'sensitive' encrypt its value in the state file? Why or why not?

Q2 What are the four main ways to supply a variable's value, and which is best for a CI/CD pipeline secret?

Deliverable: A variables.tf with at least one validation block and one sensitive variable, tested with a deliberately invalid value.

M11 Locals, Outputs & Data Sources

Goal: Compute reusable values with locals, expose results with outputs, and read existing infrastructure with data sources.

EASY EXPLANATION

locals are like local variables inside your Terraform config — computed once, used many times (e.g. a common tags map). outputs are how Terraform communicates results back out — to you, to a script, or to another Terraform module. data sources let you READ something that already exists (like an existing resource group or an existing VNet) without Terraform trying to manage or destroy it.

TOPICS TO STUDY

- locals {} for computed/derived/reused values (e.g. common tags)
- output {} for exposing values to the CLI, scripts or other configs
- data {} blocks: reading existing infrastructure Terraform does not own
- Why data sources matter: integrating with infrastructure created outside this Terraform config

HANDS-ON PRACTICAL

```
locals {
  common_tags = {
    environment = var.environment
    project     = "terraform-demo"
    managed_by  = "terraform"
  }
}

resource "azurerm_resource_group" "main" {
  name     = "rg-${var.environment}"
  location = var.location
  tags     = local.common_tags
}

output "vm_public_ip" {
  value = azurerm_public_ip.main.ip_address
}

# reading something Terraform did NOT create
data "azurerm_resource_group" "existing" {
  name = "existing-rg"
}

output "existing_rg_location" {
  value = data.azurerm_resource_group.existing.location
}
```

Interview questions

Q1 What is the practical difference between a 'locals' block and a 'variable' block?

Q2 Give a real scenario where you would need a 'data' source instead of a 'resource' block.

Deliverable: Refactor Project 4 to use a locals block for common tags applied to every resource.

PHASE 6 - Loops & Conditionals

M12 Loops: count vs for_each

Goal: Create multiple similar resources without copy-pasting blocks, and know exactly when to use count vs for_each.

EASY EXPLANATION

count is simplest when you just need N identical-ish copies of something and don't care about stable identity (e.g. 3 generic VMs). for_each is safer whenever the collection can change over time, because it tracks each instance by a stable key (a map key or set value) instead of a numeric index — so removing item #1 from the middle doesn't force Terraform to recreate everything after it.

TOPICS TO STUDY

- count = N and count.index
- for_each with a set or a map, and each.key / each.value
- Why count causes accidental resource recreation when a middle item is removed
- Industry recommendation: default to for_each for anything that can change over time

HANDS-ON PRACTICAL

```
# count: simple, index-based
resource "azurerm_linux_virtual_machine" "vm" {
  count = 3
  name  = "vm-${count.index}"
  # ...
}
# creates vm-0, vm-1, vm-2

# for_each: stable-key based (safer for changing collections)
variable "subnets" {
  type = map(string)
  default = {
    web = "10.0.1.0/24"
    app = "10.0.2.0/24"
  }
}

resource "azurerm_subnet" "this" {
  for_each             = var.subnets
  name                 = "snet-${each.key}"
  resource_group_name  = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = [each.value]
}
```

Interview questions

Q1 A subnet in the middle of a count-based list is removed. What happens to the other subnets, and why?

Q2 Why is for_each generally recommended over count for production Terraform code?

Deliverable: Convert Project 2's subnets from hard-coded blocks to a for_each over a map variable.

M13 Conditional Expressions & Dynamic Blocks

Goal: Make configuration environment-aware: different sizing for dev vs prod, and repeatable nested blocks with 'dynamic'.

EASY EXPLANATION

The ternary operator (condition ? true_value : false_value) lets one config file behave differently per environment — e.g. 1 VM in dev, 3 in prod — without duplicating files. 'dynamic' blocks do the same trick for repeated NESTED blocks (like multiple NSG security_rule blocks) instead of top-level resources.

TOPICS TO STUDY

- Ternary conditional expressions: condition ? true_val : false_val
- Using conditionals to vary instance_count, VM size, or SKU per environment
- dynamic {} blocks for repeating nested blocks driven by a list/map variable

HANDS-ON PRACTICAL

```
variable "environment" { type = string }

locals {
  instance_count = var.environment == "prod" ? 3 : 1
  vm_size        = var.environment == "prod" ? "Standard_D2s_v5" : "Standard_B1s"
}

resource "azurerm_linux_virtual_machine" "vm" {
  count = local.instance_count
  size  = local.vm_size
  # ...
}

# dynamic block example: many NSG rules from one list variable
variable "nsg_rules" {
  type = list(object({
    name     = string
    port     = string
    priority = number
  }))
}

resource "azurerm_network_security_group" "this" {
  name                = "nsg-dynamic"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name

  dynamic "security_rule" {
    for_each = var.nsg_rules
    content {
      name                       = security_rule.value.name
      priority                   = security_rule.value.priority
      direction                  = "Inbound"
      access                     = "Allow"
      protocol                   = "Tcp"
      source_port_range          = "*"
      destination_port_range     = security_rule.value.port
      source_address_prefix      = "*"
      destination_address_prefix = "*"
    }
  }
}
```

Interview questions

Q1 Write a conditional expression that sets replica count to 5 in prod and 2 everywhere else.

Q2 When would you reach for a 'dynamic' block instead of just writing the nested blocks out manually?

Deliverable: dev.tfvars and prod.tfvars producing different VM counts/sizes from the same main.tf.

PHASE 7 - Terraform State

M14 Terraform State: Concepts & Local State Commands

Goal: Understand what state actually is, why Terraform can't function without it, and practice the state inspection commands.

EASY EXPLANATION

Terraform state is Terraform's private database mapping your .tf config to real-world resource IDs. Without it, Terraform would have to guess which real object corresponds to which resource block on every run — impossible at scale. State also caches attribute values for performance and tracks dependencies.

TOPICS TO STUDY

- terraform.tfstate as the map between config and real resources
- Why Terraform requires state (identity + performance + dependency tracking)
- terraform state list / show / mv / rm / pull
- terraform refresh vs terraform plan -refresh-only

HANDS-ON PRACTICAL

```
terraform state list
terraform state show azurerm_resource_group.main
terraform state mv azurerm_resource_group.main azurerm_resource_group.core
terraform state rm azurerm_resource_group.main   # stop tracking, does NOT destroy
terraform state pull > backup.tfstate
terraform plan -refresh-only
```

Interview questions

Q1 What problem would Terraform have on every 'plan' if state did not exist?

Q2 Does 'terraform state rm' delete the real cloud resource? What actually happens?

Deliverable: Practice each state command against Project 1's resource group and document the output of each.

M15 Remote State: Azure Storage & AWS S3 Backends

Goal: Move state off your laptop into a shared, locked remote backend — mandatory for any team or CI/CD usage.

EASY EXPLANATION

A local state file breaks the moment a second person runs Terraform: two people can silently overwrite each other's state and corrupt it. A remote backend stores state centrally (Azure Storage blob, or AWS S3) and locks it during operations so only one apply can run at a time.

TOPICS TO STUDY

- Why local state fails for teams: race conditions, no locking, no shared visibility
- Azure backend: Storage Account + Blob Container holding terraform.tfstate
- AWS backend: S3 bucket for state, plus the current supported locking approach (check current AWS/Terraform docs -- native S3 locking has superseded the older DynamoDB-table pattern in recent Terraform versions)
- backend {} block and 'terraform init -migrate-state'

HANDS-ON PRACTICAL

```
# Azure remote backend
terraform {
  backend "azurerm" {
    resource_group_name  = "rg-tfstate"
    storage_account_name = "sttfstateunique"
    container_name        = "tfstate"
    key                   = "project1.terraform.tfstate"
  }
}

# AWS remote backend (verify current locking configuration against Terraform docs)
```

```
terraform {
  backend "s3" {
    bucket  = "my-tf-state-bucket"
    key     = "project1/terraform.tfstate"
    region  = "us-east-1"
    encrypt = true
    # state locking configuration: confirm the current recommended
    # approach in the Terraform documentation for your provider version
  }
}

terraform init -migrate-state
```

Interview questions

Q1 Why must remote state be locked, and what happens if two people run 'apply' at the same time without locking?

Q2 Compare the Azure Storage backend and the AWS S3 backend at a high level.

Deliverable: Migrate Project 1's state from local to a remote Azure Storage backend and verify teammates (or a second machine) can plan against it.

PHASE 8 - Modules

M16 Terraform Modules: Structure & Reuse

Goal: Package resources into reusable modules instead of copy-pasting the same VNet/VM code into every project.

EASY EXPLANATION

A module is just a directory of .tf files with its own variables (inputs) and outputs (results). Once you have a 'network' module, you call it once for dev, once for staging, once for prod, passing different variables each time — instead of maintaining three near-identical copies of the same code.

TOPICS TO STUDY

- Module anatomy: variables.tf (inputs), main.tf (resources), outputs.tf (results)
- Calling a module with the module {} block and 'source'
- Passing variables into a module, and reading a module's outputs
- Registry modules (Terraform Registry) vs local/custom modules
- Recommended repo layout: modules/ + environments/{dev,staging,prod}/

HANDS-ON PRACTICAL

```
# modules/network/variables.tf
variable "vnet_name"       { type = string }
variable "address_space"   { type = list(string) }

# modules/network/main.tf
resource "azurerm_virtual_network" "this" {
  name                = var.vnet_name
  address_space       = var.address_space
  location            = var.location
  resource_group_name = var.resource_group_name
}

# modules/network/outputs.tf
output "vnet_id" { value = azurerm_virtual_network.this.id }

# environments/dev/main.tf  (calling the module)
module "network" {
  source              = "../../modules/network"
  vnet_name           = "vnet-dev"
  address_space       = ["10.10.0.0/16"]
  location            = "East US"
  resource_group_name = azurerm_resource_group.dev.name
}
```

```
output "dev_vnet_id" {
  value = module.network.vnet_id
}
```

Interview questions

Q1 What three files does a well-structured Terraform module typically have, and what does each do?

Q2 How would you version-pin a module sourced from the public Terraform Registry?

Deliverable: Extract Project 2's networking code into a modules/network module, called from a new environments/dev folder.

M17 PROJECT 5 - Azure Multi-Environment Platform

Goal: Use your network module to stand up dev, staging and prod from the same code with different sizing per environment.

EASY EXPLANATION

This is the project where 'modules' stops being theory. You will have ONE network module and ONE compute module, called three times with three different .tfvars files, proving the same code can safely produce three different environments.

TOPICS TO STUDY

- environments/{dev,staging,prod}/ each with their own terraform.tfvars and backend key
- Sizing differences per environment via variables (VM count, size, SKU)
- Keeping each environment's state file completely separate
- A consistent tagging/naming convention across all three environments

HANDS-ON PRACTICAL

```
terraform/
  modules/
    network/
    compute/
    security/
  environments/
    dev/       main.tf   terraform.tfvars   (1 VM, small size)
    staging/   main.tf   terraform.tfvars   (2 VMs, medium size)
    prod/      main.tf   terraform.tfvars   (3+ VMs, HA)

cd environments/dev && terraform init && terraform apply -var-file=terraform.tfvars
cd ../staging && terraform init && terraform apply -var-file=terraform.tfvars
cd ../prod    && terraform init && terraform apply -var-file=terraform.tfvars
```

Interview questions

Q1 Why does each environment need its own remote state file/key, rather than sharing one?

Q2 What module design choices make it easy to add a 4th environment later without duplicating code?

Deliverable: PROJECT 5 complete: dev/staging/prod all deployed from the same modules with visibly different resource counts.

M18 PROJECT 6 - Azure Networking + Terraform Troubleshooting Lab

Goal: Combine your Azure networking expertise with Terraform by deliberately breaking routing, then diagnosing it like a real incident.

EASY EXPLANATION

This project is different: you will INTENTIONALLY misconfigure a route so that a Web VM's traffic disappears into an NVA/firewall and never reaches the internet, then use both Linux and Azure tools to find and fix it. This is the single best exercise for interview-style troubleshooting questions.

TOPICS TO STUDY

- azurerm_route_table, azurerm_route (UDR) pointing at an NVA/firewall appliance
- IP forwarding requirement on an NVA's NIC
- Diagnosing with ip route, curl, nslookup, traceroute (inside the VM)
- Diagnosing with Azure Effective Routes, NSG effective rules, NIC IP-forwarding setting

HANDS-ON PRACTICAL

```
# Terraform creates the broken path on purpose:
Web VM
  |
UDR (0.0.0.0/0 -> NVA private IP)
  |
NVA           <- IP forwarding NOT enabled (the bug)
  X
Internet

# Diagnose from inside the Web VM
ip route
curl -I https://example.com --max-time 5
nslookup example.com
traceroute example.com

# Diagnose from Azure CLI
az network nic show --ids <web-vm-nic-id> --query enableIPForwarding
az network nic list-effective-route-table --ids <web-vm-nic-id> -o table
az network nic list-effective-nsg --ids <web-vm-nic-id> -o table

# Fix: enable IP forwarding on the NVA's NIC in Terraform
resource "azurerm_network_interface" "nva" {
  # ...
  ip_forwarding_enabled = true
}
```

Interview questions

Q1 A VM can't reach the internet through an NVA. What is your step-by-step diagnostic order?

Q2 What does 'IP forwarding' on a NIC actually do, and why must it be enabled on the NVA but not the Web VM?

Deliverable: PROJECT 6 complete: documented root cause, fix and verification for a deliberately broken UDR/NVA path.

PHASE 9 - AWS Terraform Projects

M19 Azure to AWS Concept Mapping

Goal: Before writing any AWS code, map every Azure concept you already know onto its AWS equivalent.

EASY EXPLANATION

The fastest way to learn a second cloud provider is NOT to start from zero — it's to explicitly map what you already know. Almost everything you did in Azure has a direct (if not perfect) AWS equivalent.

TOPICS TO STUDY

- Azure Resource Group has no direct AWS equivalent (AWS has no top-level grouping container)
- VNet -> VPC, Subnet -> Subnet, NSG -> Security Group
- Route Table -> Route Table (same concept, same name)
- Public IP -> Elastic IP / Public IP, VM -> EC2
- Load Balancer -> ALB/NLB, NAT Gateway -> NAT Gateway (same concept, same name)
- AKS -> EKS

HANDS-ON PRACTICAL

```
Azure                AWS
-----------------    --------------------
Resource Group       (no direct equivalent - use tags/naming)
VNet                 VPC
Subnet               Subnet
NSG                  Security Group
Route Table          Route Table
Public IP            Elastic IP / Public IP
VM                   EC2
Load Balancer        ALB / NLB
NAT Gateway          NAT Gateway
IAM (Entra)          IAM
AKS                  EKS
```

Interview questions

Q1 What Azure concept has no direct 1:1 AWS equivalent, and how do AWS users typically compensate for it?

Q2 Name three AWS resource types that map almost identically in name and purpose to their Azure counterparts.

Deliverable: A personal comparison table (like above) filled in with the exact Terraform resource type names for both clouds.

M20 PROJECT 7 - AWS VPC + Subnets + Routing

Goal: Rebuild the same networking concepts from Project 2, this time in AWS, to prove the mapping is real.

EASY EXPLANATION

You already designed a VNet with public/private subnets in Azure. Now you build the AWS equivalent: a VPC with public subnets (route to an Internet Gateway) and private subnets (route to a NAT Gateway) — same concepts, different resource names.

TOPICS TO STUDY

- aws_vpc, aws_subnet, aws_internet_gateway
- aws_route_table + aws_route_table_association (public and private)
- aws_nat_gateway + aws_eip for private subnet outbound access
- aws_security_group as the AWS equivalent of an NSG

HANDS-ON PRACTICAL

```
                    VPC (10.0.0.0/16)
                          |
          +---------------+---------------+
          |                               |
   Public Subnets                 Private Subnets
   (route -> IGW)                 (route -> NAT GW)
          |                               |
      EC2 / ALB                       App EC2
```

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id
}

resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  map_public_ip_on_launch = true
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}
```

Interview questions

Q1 Why does a private subnet in AWS need a NAT Gateway to reach the internet, while a public subnet does not?

Q2 What is the AWS equivalent of an Azure NSG, and how do the rule models differ (stateful vs explicit allow/deny)?

Deliverable: PROJECT 7 complete: VPC with 1 public and 1 private subnet, correct route tables verified in the AWS Console.

M21 PROJECT 8 - AWS EC2 + User Data (Nginx) & 3-Tier Architecture

Goal: Provision an EC2 instance that self-configures with Nginx on boot, then scale the idea up to a full 3-tier design with an ALB and RDS.

EASY EXPLANATION

user_data is AWS's equivalent of Azure's cloud-init/custom data: a script that runs automatically the first time the instance boots. This lets Terraform hand off from 'create the machine' to 'do one small bootstrap task' without needing a provisioner or a separate configuration tool for something this simple.

TOPICS TO STUDY

- aws_instance with a user_data script installing and starting Nginx
- aws_security_group allowing port 80 inbound
- Scaling up: aws_lb (ALB) + target group + listener across multiple EC2s
- aws_db_instance (RDS) in a private subnet, reachable only from the app tier

HANDS-ON PRACTICAL

```
resource "aws_instance" "web" {
  ami                    = "ami-0abcdef1234567890"
  instance_type          = "t3.micro"
  subnet_id              = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.web.id]

  user_data = <<-EOF
    #!/bin/bash
    apt-get update -y
    apt-get install -y nginx
    systemctl enable nginx
    systemctl start nginx
  EOF
}

# 3-tier: Internet -> ALB -> EC2 (x2, public) -> RDS (private)
resource "aws_lb" "main" {
  name               = "app-alb"
  load_balancer_type = "application"
  subnets            = [aws_subnet.public.id, aws_subnet.public2.id]
  security_groups    = [aws_security_group.web.id]
}

resource "aws_db_instance" "main" {
  engine              = "postgres"
  instance_class      = "db.t3.micro"
  allocated_storage   = 20
  db_subnet_group_name = aws_db_subnet_group.private.name
  vpc_security_group_ids = [aws_security_group.db.id]
  skip_final_snapshot = true
}
```

Interview questions

Q1 What is 'user_data' in AWS and how does it compare to Azure's custom_data/cloud-init?

Q2 In a 3-tier design, which security group rule proves the database is unreachable from the internet?

Deliverable: PROJECT 8 complete: browser hits the ALB's DNS name and gets the Nginx page; RDS instance confirmed private-only.

PHASE 10 - Terraform + Kubernetes & Ansible

M22 Terraform + Kubernetes (AKS / EKS) and the Terraform + Ansible Boundary

Goal: Use Terraform to stand up a managed Kubernetes cluster on both clouds, and learn exactly where Terraform's job ends and Kubernetes'/Ansible's job begins.

EASY EXPLANATION

A very common mistake is trying to make Terraform manage every Kubernetes object (Deployments, Services) the same way it manages a VNet. Don't. Terraform is excellent at the INFRASTRUCTURE layer (the cluster itself, its networking, its node pools/IAM) — Kubernetes manifests/Helm/GitOps should manage what runs INSIDE the cluster. The same split applies to Ansible: Terraform creates the machine, Ansible configures what's running on it.

TOPICS TO STUDY

- azurerm_kubernetes_cluster (AKS) and aws_eks_cluster (EKS) - what Terraform manages: VPC/VNet, IAM/identity, the cluster control plane, node pools/groups
- What Kubernetes manages instead: Deployments, Services, Ingress, ConfigMaps, Secrets
- Terraform = 'create the machine', Ansible = 'configure the machine' - keep the boundary clean
- Anti-pattern to avoid: using Terraform's kubernetes/helm providers to manage every application object

HANDS-ON PRACTICAL

```
# Terraform's job: the cluster itself (illustrative, verify current arguments)
resource "azurerm_kubernetes_cluster" "aks" {
  name                = "aks-demo"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  dns_prefix          = "aksdemo"

  default_node_pool {
    name       = "system"
    node_count = 2
    vm_size    = "Standard_D2s_v5"
  }
  identity { type = "SystemAssigned" }
}

# Kubernetes' job (kubectl/Helm/GitOps), NOT Terraform:
# kubectl apply -f deployment.yaml
# helm install myapp ./chart

# Terraform's job: create the EC2 (illustrative)
resource "aws_instance" "app" { ami = "ami-xxxx" instance_type = "t3.micro" }

# Ansible's job, NOT Terraform: configure what's ON that EC2
# ansible-playbook -i inventory site.yml   (installs nginx, docker, node.js, users...)
```

Interview questions

Q1 Where exactly is the boundary between what Terraform should manage and what Kubernetes manifests/Helm should manage?

Q2 Give a one-sentence rule of thumb for deciding whether something belongs in Terraform or in Ansible.

Deliverable: A short written policy (half a page) for your own projects: 'Terraform manages X, Kubernetes manages Y, Ansible manages Z'.

PHASE 11 - Provisioners, Workspaces & Import

M23 Provisioners, Workspaces & Terraform Import

Goal: Learn three tools you will need occasionally — but that are NOT meant to be your default approach.

EASY EXPLANATION

Provisioners (local-exec, remote-exec, file) run scripts as a side effect of a resource, but they're fragile: harder to reproduce, need SSH, and aren't declarative. Workspaces let one config manage several state files, but for real dev/staging/prod separation, an environments/ folder with modules is usually clearer. Import lets you bring infrastructure someone created by hand under Terraform's management without recreating it.

TOPICS TO STUDY

- Provisioners: local-exec, remote-exec, file - and why they're a last resort (prefer cloud-init/user_data/Ansible/images)
- terraform workspace list/new/select - multiple state files from one config
- Why workspaces alone are often insufficient for large production environment separation
- terraform import: bringing existing, manually-created infrastructure under Terraform management
- Modern import {} blocks (Terraform 1.5+) as an alternative to the import CLI command

HANDS-ON PRACTICAL

```
# provisioner - last resort only
resource "aws_instance" "web" {
  # ...
  provisioner "remote-exec" {
    inline = ["sudo apt-get update", "sudo apt-get install -y nginx"]
  }
}

# workspaces
terraform workspace new dev
terraform workspace select dev
terraform workspace list

# classic import (existing EC2 -> Terraform state)
terraform import aws_instance.web i-0abcd1234efgh5678

# modern import block (Terraform 1.5+, verify current syntax)
import {
  to = aws_instance.web
  id = "i-0abcd1234efgh5678"
}
```

Interview questions

Q1 Why are provisioners considered a 'last resort' rather than a default deployment mechanism?

Q2 Walk through the steps to bring an EC2 instance that a colleague created manually in the AWS Console under Terraform management.

Deliverable: Successfully import one manually-created test resource (e.g. an S3 bucket) into a fresh Terraform config and confirm 'terraform plan' shows no changes.

PHASE 12 - Debugging, Security & CI/CD

M24 Debugging, Security & Secrets Management

Goal: Build a systematic debugging checklist and lock down secrets so nothing sensitive ever lands in Git or plain output.

EASY EXPLANATION

Most Terraform failures fall into a small number of buckets: authentication, invalid input (like a bad CIDR), a resource dependency Terraform can't resolve, a state/reality mismatch, or a cloud-side limit (quota, permissions, region). Debug in that order before assuming something exotic is wrong. Separately: never hardcode secrets — use variables, environment variables, or dedicated secret managers.

TOPICS TO STUDY

- Systematic debugging order: auth -> invalid input -> dependency -> state mismatch -> quota/permissions/region
- TF_LOG=DEBUG terraform plan for verbose provider/API logs
- Never commit secrets: use variables, environment variables, or a secret manager
- Azure: Key Vault + Managed Identity + RBAC
- AWS: Secrets Manager, SSM Parameter Store, IAM least privilege

HANDS-ON PRACTICAL

```
# systematic checklist when 'terraform apply' fails
# 1. Provider authentication      -> az login / aws sts get-caller-identity
# 2. Invalid input                -> check CIDR, region, name uniqueness
# 3. Resource dependency          -> terraform graph | less
# 4. State mismatch               -> terraform plan -refresh-only
# 5. Quota / permissions / region -> check IAM policy / Azure RBAC / service quota

TF_LOG=DEBUG terraform plan 2> debug.log

# NEVER do this:
# password = "mypassword123"

# instead:
data "azurerm_key_vault_secret" "db_password" {
  name         = "db-password"
  key_vault_id = azurerm_key_vault.main.id
}

data "aws_secretsmanager_secret_version" "db_password" {
  secret_id = "prod/db/password"
}
```

Interview questions

Q1 Walk through your debugging order for a Terraform apply that fails with a generic 'permission denied' error.

Q2 Compare Azure Key Vault + Managed Identity with AWS Secrets Manager + IAM roles for keeping secrets out of code.

Deliverable: A written debugging checklist (half a page) you can genuinely follow during an outage, plus one variable migrated from hardcoded to Key Vault/Secrets Manager.

M25 Terraform CI/CD, Linting & Security Scanning

Goal: Wire Terraform into a GitHub Actions pipeline with formatting, validation, linting and security scanning before every apply.

EASY EXPLANATION

In a real team, nobody runs 'terraform apply' from their laptop against production. A pull request triggers fmt, validate, a linter (TFLint), a security scanner (Checkov or tfsec/Trivy) and a plan that a human reviews and approves — only then does apply run, usually from the pipeline itself with tightly scoped credentials.

TOPICS TO STUDY

- Pipeline stages: fmt -> validate -> TFLint -> Checkov/tfsec -> plan -> approval -> apply
- TFLint for Terraform-specific linting (naming, deprecated syntax, provider-specific rules)
- Checkov / tfsec / Trivy for infrastructure security and policy scanning
- Posting the plan output as a PR comment for human review before approval
- Repository layout: modules/, environments/, .github/workflows/terraform.yml

HANDS-ON PRACTICAL

```
# .github/workflows/terraform.yml (illustrative, verify current action versions)
name: Terraform
on: [pull_request]
permissions: { id-token: write, contents: read, pull-requests: write }
jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform fmt -check -recursive
      - run: terraform init
      - run: terraform validate
      - uses: terraform-linters/setup-tflint@v4
      - run: tflint --recursive
      - run: pip install checkov && checkov -d .
      - run: terraform plan -out=tfplan
      # post-plan: comment on PR, require manual approval before apply job runs
```

Interview questions

Q1 Why should 'terraform apply' for production typically run from a pipeline rather than a developer's laptop?

Q2 What is the difference between what TFLint checks and what Checkov/tfsec check?

Deliverable: A working GitHub Actions workflow that runs fmt/validate/TFLint/Checkov/plan on every pull request to your Terraform repo.

PHASE 13 - Capstone

M26 CAPSTONE - Multi-Cloud DevOps Infrastructure Platform

Goal: Bring every phase together: one GitHub repo, modules for both clouds, CI/CD with security scanning, deploying AKS and EKS side by side.

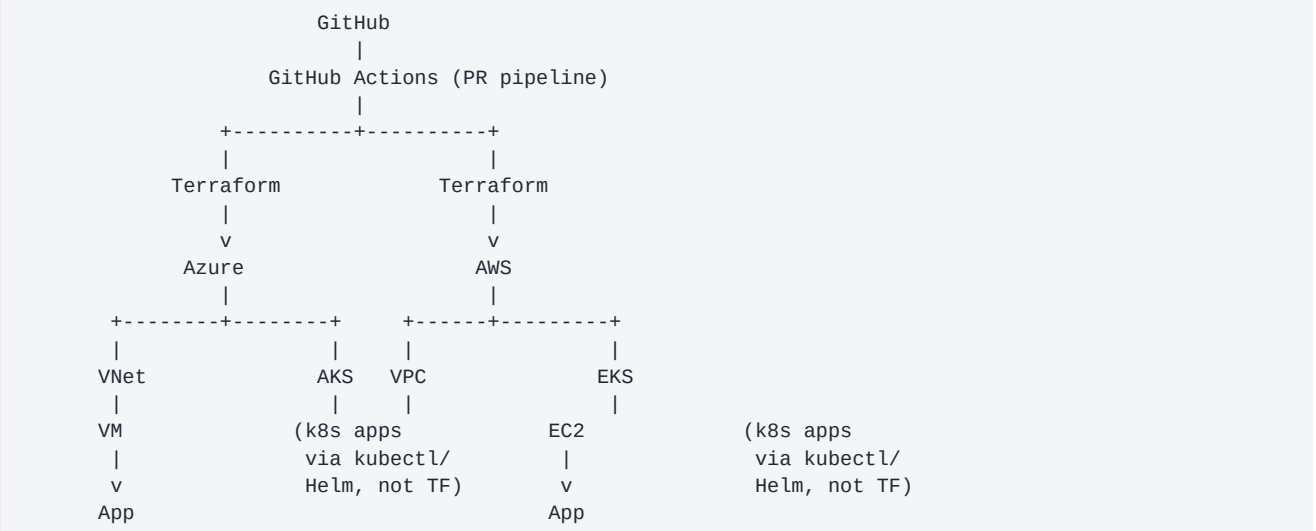
EASY EXPLANATION

This is the project that proves you can operate as a platform engineer, not just someone who knows Terraform syntax. The same PR-based workflow, the same module patterns, and the same security scanning apply whether the target is Azure or AWS — that consistency IS the skill.

TOPICS TO STUDY

- Repository layout: modules/{network,compute,security,load-balancer,kubernetes,database}/
- environments/{dev,staging,prod}/ for BOTH azure/ and aws/ targets
- GitHub Actions: PR -> fmt -> validate -> security scan -> plan -> approval -> apply, for each cloud
- AKS and EKS deployed side by side from the same repo using shared conventions
- A written architecture decision record (ADR) explaining why each cloud-specific choice was made

HANDS-ON PRACTICAL




```
modules/  network/ compute/ security/ load-balancer/ kubernetes/ database/
environments/ azure/{dev,staging,prod}/  aws/{dev,staging,prod}/
```

Interview questions

Q1 Walk an interviewer through this architecture end to end, including why Kubernetes objects are NOT managed by Terraform.

Q2 What would you change about this design to support a THIRD cloud provider in the future?

Deliverable: CAPSTONE complete: a public GitHub repo with modules, both cloud environments, a green CI/CD pipeline, and a README with architecture diagrams.

Interview Question Bank (with Model Answers)

Use these to self-test after finishing the modules. Cover the answer, ask yourself the question out loud, then check.

Core Concepts

Q: What is Terraform and why use Infrastructure as Code?

A: Terraform is a declarative IaC tool: you describe desired infrastructure state in files, and Terraform computes and executes the API calls needed to reach that state. IaC gives you version control, repeatability, peer review and safe collaboration on infrastructure — instead of manual, undocumented changes.

Q: Terraform vs Ansible vs CloudFormation - how do you compare them?

A: Terraform: declarative, multi-cloud, strong state management, best at provisioning infrastructure. Ansible: primarily imperative/procedural, agentless, best at configuring servers after they exist. CloudFormation: declarative like Terraform but AWS-only, tightly integrated with AWS services but not portable to other clouds.

Q: What is a Terraform provider, and how is it different from a resource?

A: A provider is a plugin that lets Terraform talk to a specific API (azurerm, aws, docker, github...). A resource is a specific object you ask that provider to create/manage (a VM, a bucket, a VNet).

Q: What is a data source and when would you use one?

A: A data source reads information about infrastructure that already exists, without Terraform trying to manage or destroy it — e.g. looking up an existing resource group's location to use in a new resource.

Variables, Loops & Expressions

Q: count vs for_each - when do you use each?

A: count is simplest for a fixed number of near-identical resources where you don't need stable identity. for_each (over a set or map) is safer whenever the collection can change over time, because each instance is tracked by a stable key instead of a numeric index, avoiding unnecessary recreation.

Q: What variable types does Terraform support?

A: string, number, bool, list, set, map, object and tuple, plus validation blocks and the sensitive flag.

Q: How do you write a conditional expression in Terraform?

A: Using the ternary operator: condition ? true_value : false_value — for example, var.environment == "prod" ? 3 : 1.

Q: What is a dynamic block and when would you use it?

A: A dynamic block repeats a NESTED block (like an NSG security_rule) based on a list/map variable, avoiding manually duplicating many similar nested blocks inside a single resource.

State & Backends

Q: What is Terraform state and why is it necessary?

A: State is Terraform's record mapping your configuration to real-world resource IDs. Without it, Terraform couldn't know which real object corresponds to which resource block, track dependencies, or compute an accurate diff on each plan.

Q: Why use remote state instead of local state?

A: Local state breaks the moment more than one person or pipeline runs Terraform: no locking means concurrent runs can corrupt or overwrite state. Remote backends (Azure Storage, AWS S3) provide a shared, lockable, centrally accessible state store.

Q: What is state locking, and what happens if it fails?

A: State locking prevents two operations from writing to state at the same time. If a lock can't be acquired, Terraform refuses to proceed, protecting against corruption; a stuck lock can be manually cleared with terraform force-unlock.

Q: How do you import existing infrastructure into Terraform?

A: Using 'terraform import <resource_address> <cloud_id>' to associate an already-existing real resource with a resource block you've written, or with the modern 'import {}' configuration block — then you still need to write matching .tf configuration so future plans show no changes.

Modules & Design

Q: What is a Terraform module and why use one?

A: A module is a directory of .tf files with its own inputs (variables) and outputs, letting you reuse the same resource definitions across multiple environments or projects by simply changing input values.

Q: How do you manage dev/staging/prod with Terraform?

A: Commonly via an environments/{dev,staging,prod}/ folder structure, each calling shared modules with environment-specific variable values and its own remote state file — rather than relying solely on Terraform workspaces for large production separation.

Q: Are Terraform workspaces the same as separate environments?

A: Not exactly — workspaces let one configuration manage multiple state files, which works for small variations, but most teams find a modules + environments folder structure clearer and safer for genuinely different environments (different sizing, different approval processes, etc.).

Security & Operations

Q: How do you handle secrets in Terraform?

A: Never hardcode them in .tf files. Use variables marked sensitive, environment variables (TF_VAR_*) in CI/CD, or read them from a dedicated secret manager (Azure Key Vault, AWS Secrets Manager/SSM Parameter Store) via a data source at apply time.

Q: How do you troubleshoot a failing Terraform apply?

A: Work through a systematic order: provider authentication, invalid input (bad CIDR, wrong region), resource dependency issues, state/reality mismatches, then cloud-side limits like quota or IAM permissions.

Q: How do you run Terraform safely through CI/CD?

A: A pull request triggers fmt, validate, a linter (TFLint) and a security scanner (Checkov/tfsec/Trivy), then a plan that a human reviews; only after approval does an apply job run, ideally with tightly scoped, short-lived credentials rather than a developer's personal credentials.

Q: What's the difference between what TFLint and Checkov/tfsec check?

A: TFLint focuses on Terraform-specific correctness: deprecated syntax, provider-specific best practices, naming conventions. Checkov/tfsec focus on security and compliance: overly permissive security groups, unencrypted storage, missing logging, public exposure, etc.

Cloud & Architecture

Q: Build the same 3-tier architecture on Azure and AWS - what maps to what?

A: VNet/VPC for networking, Subnet/Subnet for segmentation, NSG/Security Group for firewalling, VM/EC2 for compute, Azure Load Balancer or App Gateway/ALB or NLB for load balancing, and AKS/EKS if the tier runs on Kubernetes.

Q: Where is the line between what Terraform manages and what Kubernetes manages?

A: Terraform manages the infrastructure layer: the cluster itself, networking, IAM/identity, node pools/groups. Kubernetes manifests, Helm or GitOps manage what runs INSIDE the cluster: Deployments, Services, Ingress, ConfigMaps, Secrets. Mixing these responsibilities makes both harder to reason about.

Q: Terraform vs Ansible in a real pipeline - how do they work together?

A: Terraform provisions the infrastructure ('create the machine' — the VM, network, security group). Ansible (or similar) then configures what runs on that machine (packages, users, application deployment). Keeping this split clean avoids duplicating responsibility between two tools.

Final Tips Before You Start

- Always run terraform plan and read it fully before terraform apply — plan is your safety net.
- Never commit terraform.tfstate, *.tfstate.backup, or *.tfvars files containing secrets.
- Default to for_each over count for anything that can change over time.
- Keep Terraform's job (infrastructure) and Kubernetes/Ansible's job (what runs on/in it) clearly separated.
- Move to remote state as soon as more than one person touches a project — don't wait for a real incident to force it.
- Build something for every module. Reading alone will not prepare you for an interview or a real job.

Next step: Start with Module 0 today. By the time you reach the Capstone, you will be able to stand up the same architecture on Azure and AWS from memory — that dual-cloud fluency is what turns Terraform from 'a tool I've used' into a genuine platform engineering skill.